PRODUCT REQUIREMENT DOCUMENT (PRD)

SunAI – EyeVen

PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026
Project Name: SunAI – EyeVen
Product: AI Vendor Onboarding & Compliance Orchestrator
Version: 1.1 – Updated to match GitHub branch `initial-eyeven-project`

Table of Contents

PRODUCT REQUIREMENT DOCUMENT (PRD) 1
1. Project Overview 3
2. Background & Business Objective 3
3. Product Purpose 4
3.1 Main Goal of the System 4
3.2 Intended Users 4
4. System Functionalities 5
4.1 Description 5
4.2 Key Functionalities 5
4.3 AI Model & Prompt Design 5
4.4 Workflow and Data Scope 6
4.5 Technology Stack 6
5. User Stories & Use Cases 7
5.1 User Stories 7
5.2 Main Use Cases 7
5.3 Use Case Diagram Description 8
6. Features Included (Scope Definition) 8
7. Features Not Included (Scope Control) 10
8. Assumptions & Constraints 11
9. Risks & Questions Throughout Development 11
Appendix A: Source Traceability Summary 12

1. Project Overview

Problem Statement: Vendor onboarding submissions are often received as messy document packages containing PDFs, scanned images, spreadsheets, CSV files, ZIP bundles, and optional email content. Procurement teams must manually inspect documents, identify vendor details, compare information, verify legitimacy, assess risk, and decide whether a case should be completed, followed up, rejected, flagged, or reviewed manually. This is slow, repetitive, inconsistent, and difficult to audit.

Target Domain: EyeVen is positioned in enterprise procurement, vendor onboarding, and compliance management. The system focuses on the early onboarding stage where vendor information must be converted into a structured case before procurement decisions can be made.

Proposed Solution Summary: EyeVen is an AI-powered vendor onboarding and compliance orchestration system. The implementation allows users to upload vendor files, create vendor cases, run a multi-stage AI workflow, review extracted canonical form fields, inspect verification findings, view policy and risk assessment results, review routing decisions, generate follow-up email drafts, open the draft through Outlook using mailto integration, upload follow-up files, reprocess cases, and monitor case status through a dashboard.

2. Background & Business Objective

Vendor onboarding is commonly fragmented across emails, files, spreadsheets, and manual checking. When submissions are incomplete or inconsistent, procurement officers must manually contact vendors, request clarifications, and repeat review after corrected documents arrive. EyeVen improves procurement speed, consistency, and traceability by converting unstructured submissions into structured, risk-aware, auditable cases.

EyeVen aligns with the hackathon focus on AI systems and agentic workflow automation because it demonstrates multi-step reasoning over unstructured inputs, structured JSON output validation, workflow orchestration, case-state management, follow-up generation, and reprocessing after corrected submissions.

3. Product Purpose

3.1 Main Goal of the System

The main goal of EyeVen is to streamline vendor onboarding by transforming messy vendor submissions into structured, verifiable, risk-aware, and reviewable procurement cases. The system supports the procurement team from initial document upload until routing decision, follow-up generation, and reprocessing of updated documents.

3.2 Intended Users

- **Procurement Officer:** uploads submissions, creates cases, runs workflows, reviews extracted fields, checks routing decisions, generates follow-up emails, and uploads corrected documents.
- **Compliance Officer:** reviews verification findings, risk bands, blocking issues, recoverable issues, and audit trail information.
- **Procurement / Operations Team:** monitors case status, risk distribution, follow-up cases, and completed cases through dashboard and case list views.
- **Senior Approver / Management:** reviews summarized case details and high-risk routing results when escalation is needed.

4. System Functionalities

4.1 Description

EyeVen operates as a web-based vendor onboarding workflow system with a React TypeScript frontend and Flask backend. The frontend provides Upload, Procurement Dashboard, Vendor Cases, and Case Detail pages. The backend exposes REST API routes for cases, uploads, workflow execution, dashboard analytics, and audit history. Data is stored in MySQL.

4.2 Key Functionalities

- Case creation with auto-generated reference, optional vendor name, optional email content, and uploaded files.
- Drag-and-drop and browse-based upload for PDF, XLSX, XLS, CSV, PNG, JPG, JPEG, and ZIP files.
- File validation, secure stored filenames, metadata recording, and SHA-256 checksum.
- Extraction pipeline for ZIP extraction, deterministic CSV/XLSX parsing, and PaddleOCR-based OCR for PDF/image files.
- Intake classification to detect procurement-form-present or procurement-form-absent submissions.
- Canonical EyeVen form structuring using GLM and seeded vendor fields.
- Form alignment review with value, source, status, confidence score, and notes.
- Verification agent using internal registry matching and external verification signals.
- Policy evaluation agent producing risk band, blocking issues, recoverable issues, risk flags, and summary.
- Routing agent producing complete_verified, follow_up_required, flagged_high_risk, rejected, or manual_review_required decisions.
- Follow-up email drafting with Outlook/mailto handoff for human review before sending.
- Follow-up file upload and reprocessing for pending_follow_up cases.
- Audit trail for verification, policy, routing, and reprocessing runs.
- Dashboard analytics for total cases, pending follow-up, high-risk cases, completed cases, status distribution, and risk distribution.
- Searchable Vendor Cases list.

4.3 AI Model & Prompt Design

The main reasoning model is Z AI GLM, configured through an OpenAI-compatible chat completion API. It is used for canonical form structuring, verification reasoning, policy evaluation, routing decisions, and follow-up email generation. EyeVen uses staged prompting rather than one monolithic prompt. Each stage returns schema-validated JSON output that can be safely persisted and passed forward.

Uploaded documents are preprocessed before LLM use. Spreadsheet files are parsed locally, while PDFs and images are OCR-processed. The GLM service truncates overly long prompt content to a maximum context character limit, parses JSON, attempts limited repair, validates schema, and falls back to controlled states when failures occur.

4.4 Workflow and Data Scope

Area	Implemented Details
Frontend routes	`/`, `/dashboard`, `/cases`, `/cases/:caseId`
Backend APIs	`/api/cases`, `/api/uploads`, `/api/workflow`, `/api/dashboard`, `/api/audit`
Workflow states	uploaded, intake_received, extracting, intake_classified, field_alignment_completed, incomplete_after_autofill, verification_in_progress, policy_review_in_progress, pending_follow_up, reprocessing, manual_review_required, completed, rejected, flagged_high_risk
Database tables/models	vendor_cases, uploaded_files, eyeven_form_field_values, verification_runs, policy_evaluation_runs, routing_decisions, email_drafts, reprocessing_runs, vendor_registry

4.5 Technology Stack

Layer	Technology	Purpose
Frontend	React, TypeScript, Material UI	Interface and routing
Charts	Chart.js, react-chartjs-2	Dashboard charts
Backend	Python, Flask, Flask-CORS	REST API and workflow orchestration
Database	MySQL, SQLAlchemy, Flask-Migrate, PyMySQL	Persistence
AI	Z AI GLM	Structured reasoning
OCR	PaddleOCR, PaddlePaddle, pdf2image, Pillow	PDF/image extraction
Parsing	openpyxl, CSV	Spreadsheet extraction
Validation	jsonschema, marshmallow	Structured output validation
Deployment/dev	Docker, docker-compose, GitHub	Development and container support

5. User Stories & Use Cases

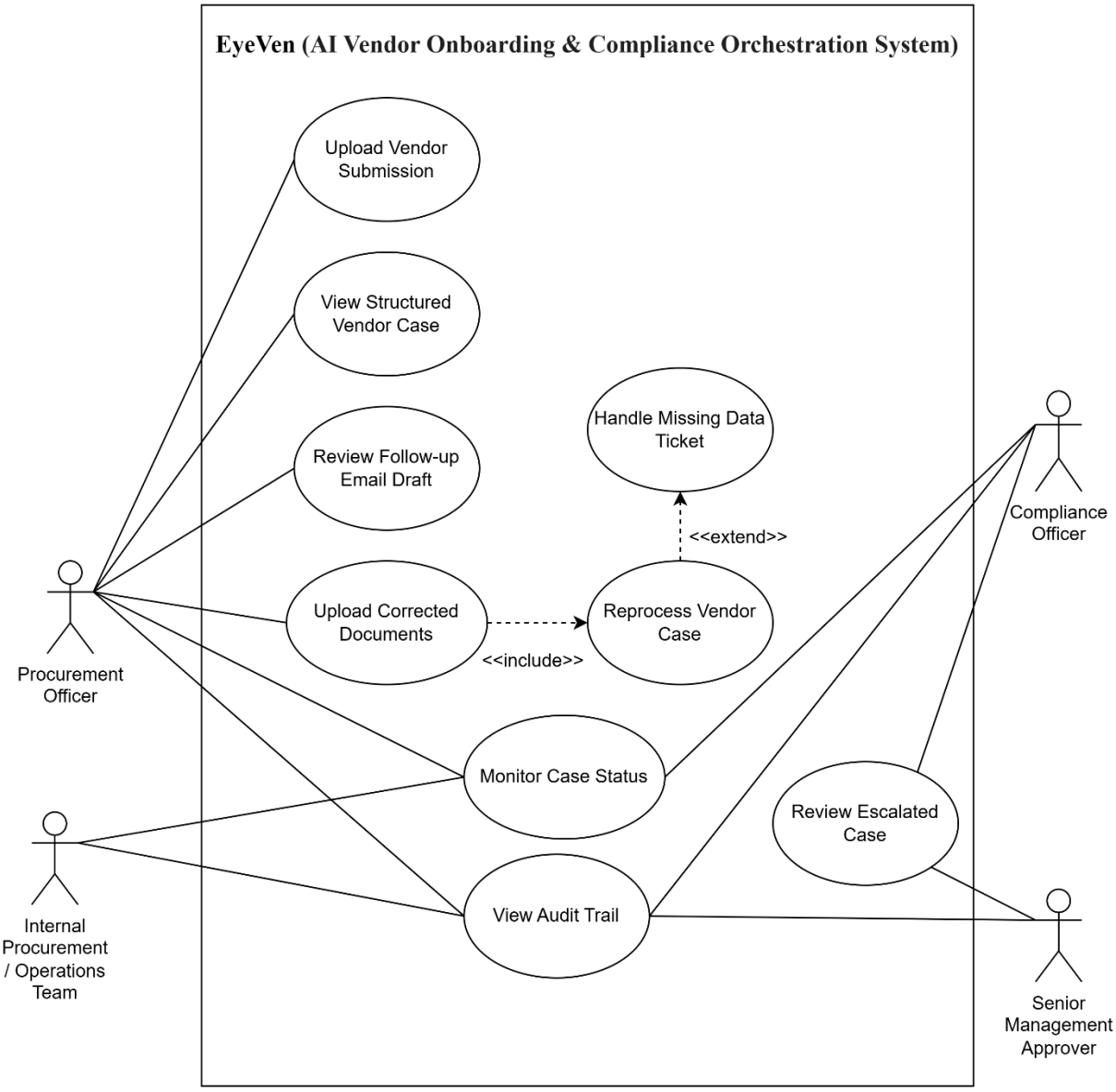
5.1 User Stories

- As a procurement officer, I want to upload vendor submission files so EyeVen can create a structured onboarding case.
- As a procurement officer, I want to run an AI workflow so vendor documents can be extracted, verified, assessed, and routed.
- As a compliance officer, I want to view verification findings and policy risk flags so risky or incomplete vendors are not approved accidentally.
- As a procurement officer, I want to generate a follow-up email draft so missing information can be requested consistently.
- As a procurement officer, I want to upload follow-up documents into the same case so corrected submissions can be reprocessed without duplicate cases.
- As an operations user, I want dashboard summaries to monitor case volume, case status, and risk distribution.
- As a reviewer, I want an audit trail so I can understand how decisions were reached.

5.2 Main Use Cases

Use Case	Implemented Support
Vendor submission intake	Upload page, case creation API, upload API
Extraction and structuring	Extraction orchestrator, OCR, parser, form service
Verification	Verification agent, internal registry, external verification service
Policy and risk assessment	Policy evaluation agent and panel
Routing	Routing agent and routing panel
Follow-up email	Follow-up endpoint, email draft storage, Outlook mailto
Corrected submission reprocessing	Follow-up upload and reprocessing orchestrator
Monitoring	Dashboard, case list, audit trail

5.3 Use Case Diagram Description



The diagram should show Procurement Officer as the main actor who uploads submissions, creates cases, runs workflows, reviews case details, generates follow-up emails, opens drafts in Outlook, uploads follow-up files, and reprocesses cases. Compliance Officer and Senior Approver inspect verification, policy/risk, routing, and audit outputs.

6. Features Included (Scope Definition)

- React TypeScript frontend with Material UI navigation for Upload, Dashboard, and Cases.
- Upload page with drag-and-drop upload, vendor display name, optional email content, file preview, and post-upload Run Workflow action.

- Case creation, list, detail, workflow run, rerun, follow-up, dashboard, and audit APIs.
- Supported uploads: PDF, XLSX, XLS, CSV, PNG, JPG, JPEG, ZIP.
- File metadata persistence with checksum and storage path.
- Extraction pipeline using ZIP extraction, spreadsheet parsing, and PaddleOCR.
- Canonical vendor form field seeding and GLM-updated field values.
- Schema-validated GLM stages: canonical form, verification, policy evaluation, routing, follow-up.
- Internal vendor registry matching with high-risk history and multi-vendor mismatch detection.
- External verification service with SST portal scraping behavior in demo implementation.
- Case Detail panels for form alignment, verification, policy/risk, routing, follow-up, reprocessing, and audit trail.
- Dashboard analytics and searchable case list.
- Follow-up email draft generation and Outlook/mailto handoff.
- Follow-up document upload and reprocessing.
- MySQL schema/models for core case and workflow records.
- Docker Compose support for MySQL and backend.

7. Features Not Included (Scope Control)

- Authentication, login, registration, role-based access control, and multi-user permission enforcement are not implemented in the current EyeVen branch.
- Separate role-based dashboards for procurement officer, compliance officer, senior management, or admin are not implemented.
- Direct email sending is not implemented; the system opens the user email client through mailto.
- Full production-grade SSM live verification is not implemented because CAPTCHA blocks automation.
- Full LHDN live verification is not implemented.
- Human approval assignment, comments, and approval sign-off workflow are not implemented.
- Ticketing integrations such as Jira, ServiceNow, or Zendesk are not implemented.
- Production cloud deployment is not fully completed.
- Advanced security controls such as JWT, role-based API authorization, antivirus scanning, and secret rotation are not completed.
- Automated tests are not documented as completed.
- Frontend editing of extracted fields is not implemented.

8. Assumptions & Constraints

- Each full workflow may call GLM multiple times, so cost depends on document size and whether follow-up drafting is triggered.
- The system controls prompt size by local preprocessing and truncating long prompt content.
- OCR quality depends on scan quality, image clarity, and PaddleOCR performance.
- SST portal scraping depends on external website availability, page structure, network stability, ChromeDriver compatibility, and anti-bot limitations.
- MySQL and GLM API configuration must be available through environment variables or Docker setup.
- The current implementation is suitable for prototype/demo validation rather than production enterprise deployment.
- Follow-up reprocessing is only available for cases in pending_follow_up state.
- Large PDFs, image-heavy files, and ZIP bundles increase processing time.

9. Risks & Questions Throughout Development

Risk / Question	Impact	Mitigation / Current Handling
Malformed GLM output	Breaks workflow persistence	JSON parsing, repair attempt, schema validation, fallback states
OCR inaccuracies	Wrong or missing fields	Confidence/status display and follow-up reprocessing
External portal instability	Verification failure	Fallback behavior and strict non-trust of format-only checks
No authentication/RBAC	Data exposure in production	Out of current demo scope; required before production
AI hallucination	Unsupported vendor facts	Prompts instruct no invention; schema supports missing/conflicting/unverifiable
Sensitive file handling	Confidentiality risk	Production needs secure storage, access control, scanning, retention policy
No direct email send	Requires manual action	Human review through Outlook/mailto handoff
Scalability	Slow OCR/LLM/Selenium under load	Future queues, async jobs, caching, and workers

Appendix A: Source Traceability Summary

This PRD was updated to match GitHub repository `Lyann444/EyeVen`, branch `initial-eyeven-project`, and the uploaded PRD draft. The branch confirms the implemented frontend routes, backend routes, services, workflow orchestration, schema validation, database models, and file processing pipeline. Anything not present in the final branch has been moved to Features Not Included or future-scope wording.